Listen

Lab 12: Operating System Job Scheduler

30 points · Unit 7 · Due Sunday, November 8, 11:59 PM

The largest lab in the course, and the first one that is a *simulation* rather than a data structure exercise. You are building a CPU scheduler on top of a priority queue and then running it a thousand times to find out how it behaves.

That second half is what makes this lab different. A single run tells you nothing, because the job arrivals and lengths are random. Only the distribution across many runs answers the question the specification actually asks — and learning to distinguish those two things is worth as much as the code.

The priority queue is a heap. Getting the highest-priority job in O(log n) rather than scanning a list is precisely why an operating system uses this structure.

Specification

Priority queues are used by operating systems to schedule jobs on a CPU. Each iteration of the scheduler corresponds to one CPU cycle; assume each cycle takes one nanosecond.

A job has a name (e.g. `P1`), a **priority** (an integer from −19 to 20, where 20 is highest), and a **length** (an integer from 1 to 100, the number of cycles needed to complete it).

A job cannot be interrupted. Once scheduled on the CPU it runs for a number of cycles equal to its length. During each cycle the CPU will process its current job, add new jobs to the queue, wait for another job to be scheduled, or terminate.

Simulation parameters

- The simulator processes **50 jobs**, each with random priority and length.
- Job completion takes approximately 1 to 100 cycles.
- Inter-arrival time is **20 cycles**. Simulate by generating a random 0 or 1 every 20 cycles — 0 admits no new job, 1 admits one. Effectively a coin flip.
- Run the simulator over a span of **2,700 cycles**.
- Run **1,000 simulations per test**, across no more than **5 attempts**, to find a benchmark threshold that lets all 50 jobs complete in the shortest time. You may not need all 2,700 cycles.

Required output

Compute the average number of jobs in the heap per cycle by tallying the number of jobs in the heap at each clock cycle and dividing by the total number of cycles required. After each simulation:

```
Average number of jobs in the heap per cycle: 4
  The scheduler completed all 50 jobs.
  It took 2700 cycles to complete them.

      OR

  Average number of jobs in the heap per cycle: 3
  The scheduler did not complete all 50 jobs.
  Jobs processed: 49
```

And after each test of 1,000 simulations:

```
Completed simulations: 939
  Incomplete simulations: 61
```

Required classes

```
class Job{
  public:
      Job();
      Job(int p, int l);
      int getLength() const;
      void decrementLength();
      int getPriority() const;
      friend ostream& operator<<(ostream& out, const Job& j);
      bool operator<(Job otherJob);
      bool operator>(Job otherJob);
      bool operator==(Job otherJob);
      bool operator<=(Job otherJob);
  private:
      int length;
      int priority;
  };
```

```
template <class ItemType>
  class Scheduler: public PQType<ItemType>{
  public:
      Scheduler(int s);
      int getNumJobs() const;
      void setCurrentJob(ItemType& j);
      void addJob(ItemType& j);
      void removeJob(ItemType& j);
```

```
private:
    int numJobs;
    ItemType currentJob;
};

template <class ItemType>
Scheduler<ItemType>::Scheduler(int size) : PQType<ItemType>(size){
    numJobs = 0;
}
```

Build a `Job` class, a `Scheduler` class, and a separate driver to manage the simulation. Use the provided `PriorityQueue.zip`.

Files Provided

> **Download `Lab 12 Files`** — it is the item directly below this document in Unit 7. Everything listed here is in that one zip. Unzip it into a single folder and compile there.

- `PriorityQueue.zip` — the `PQType` priority queue built on a heap. **Start here.**
- `CSC240_Heap_Practice.zip` — supplemental. Heap examples for reference.

AI Learning Query

Have this conversation while you work. Start with the prompt below, continue for at least five exchanges, paste your own code in at least once, and push back at least once when an answer seems wrong or incomplete. This one is **not submitted**. It is here because it will make the lab easier, and because the habit is the point.

> *"I am simulating a CPU job scheduler in C++ on top of a heap-based priority queue, and I have to run it 1,000 times and report how often all 50 jobs finish. Do not write the simulation. Help me think about the experiment design: what exactly should I measure per run, how do I decide whether a difference between two attempts is real or just randomness, and what does it mean to report a 'reliable' completion rate from 1,000 trials? I want to understand the statistics well enough to defend my threshold."*

STL Counterpart

Ungraded — read this, do not submit it. `std::priority_queue` is the heap you are building on, and it is worth reading its interface to see that it exposes only `top`, `push`, and `pop` — no way to inspect or modify a queued element, which is exactly why aging is hard to add. Also look at `<random>`: `std::mt19937` with a distribution is a far better generator than `rand()` for a study running a million trials. Your submission uses the provided `PQType`.

Be Ready to Answer

This lab has no Mini Oral. Question 1 is submitted in writing with your code and is graded under Design and Documentation; the rest are not submitted, but all of them are fair game in your Major Oral Inquiry.

Every question below is about *your* submission. You may use AI to prepare — that is encouraged — but the answers must describe the code you actually wrote.

1. State the invariant your heap maintains after every insertion and removal, and name the operation in *your* code that restores it. Write this as a comment.
2. The specification asks for a “benchmark threshold.” Say exactly what yours is — minimum cycles, minimum completion probability, minimum average heap size, or something else — and defend why that is the right definition of the question being asked.
3. You ran up to 5 attempts of 1,000 simulations. Report your completion rates. Did they vary meaningfully between attempts? What does the amount of variation tell you about the system?
4. Give an informal confidence claim from your data, in the form “with X cycles, completion occurs about Y% of the time.” Then say what *you* would call reliable, and why.
5. Jobs cannot be interrupted. Using your own measurements, state the effect this has on (a) the average number of jobs in the heap per cycle and (b) the probability of finishing all 50 jobs within 2,700 cycles.

Grading

Criterion	What earns the points	Pts
Correctness and Specification	Every requirement in the specification is met and demonstrated by the driver. Output matches what is stated above. Edge cases named in the specification behave correctly.	**15**

Data Structure Implementation	The required structure is implemented as specified. No STL container is substituted for work the specification asks you to do yourself. Memory is allocated and released correctly.	**8**
Design and Documentation	Interface separated from implementation. Preconditions and postconditions documented on every public function you write or change.	**4**
Coding Conventions	Follows the Oakton C++ Coding Conventions: naming, indentation, commenting, file organization.	**3**
Required work subtotal		**30**
Total		30

Submitting. Upload all source files plus your driver to the Lab 12 dropbox. **Late submissions are not accepted** — the dropbox closes at the due date. Office hours are virtual, 30-minute slots, Monday through Friday 7:00–8:00 PM; the booking link is in *Start Here*.